

AI WeatherWise

Deliver Intelligent Weather Forecasts And Insights.

Team Members

Jothika.R

Dhana Lakshmi.S Bhavana.G

Dhana Lekshmi.S

Nithya Sree.P

Project Description:

AI WeatherWise is a simple weather application. It shows current weather information and gives helpful suggestions. The app is built using Node.js, Express.js, MongoDB, and Gemini AI.

The app shows weather details, gives short AI-based weather summaries, and suggests activities or clothing based on the weather.

Users can create an account, save their favorite cities, and check the weather whenever they want.

Scenario-Based Case Study:

Background:

Alex enjoys travelling to different cities. He wants an easy way to check the weather and get suggestions for activities.

Problem:

- It is hard to manage weather information for many cities.
- Users may not know what activities or clothes suit the weather.
- Raw weather numbers can be hard to understand.

- Users need a safe way to save their favorite cities.

Solution:

AI WeatherWise collects weather data and uses Gemini AI to create simple summaries and suggestions. Users can save their favorite cities and check the weather anytime.

Usage:

- Users can add, view, update, and delete favorite cities.
- Users can check the current weather.
- Users can ask for AI-based weather suggestions.
- Admins can check the system and API status.
- Weather data can also be accessed through public APIs.

Outcome:

The app makes weather information easy to understand. It also gives useful suggestions based on the current weather.

Software Requirements:

- Operating System: Windows 10/11, macOS or Linux.
- Node.js: Runs the backend application.
- npm: Installs and manages packages.
- Gemini AI: Creates weather summaries and suggestions.
- MongoDB: Stores user and location data.
- Postman: Tests the APIs.
- Visual Studio Code: Used to write and edit code.

Project Architecture:

AI WeatherWise follows the MVC (Model-View-Controller) pattern. This pattern splits the app into different parts so it is easier to understand and maintain.

Main Components:

- API Interface: Sends requests and receives weather data.
- Express Server: Handles requests and routes.

- Authentication: Checks if the user can access protected features.
- Services: Holds the main app logic.
- Database Layer: Uses Mongoose to talk to MongoDB.
- AI Service: Uses Gemini AI to create summaries and suggestions.

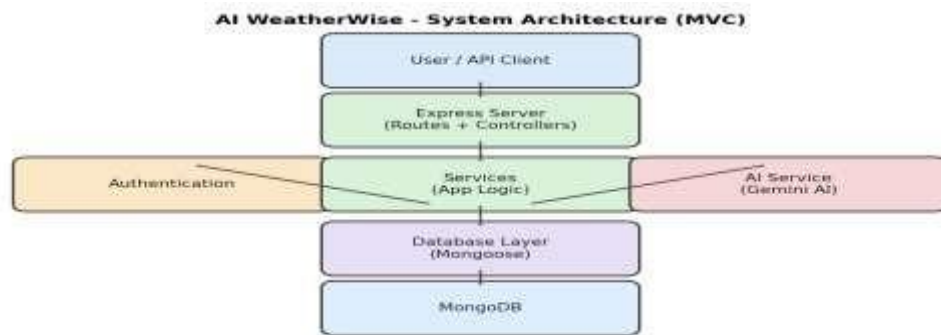


Figure 1: System Architecture (MVC)

Database / ER Diagram:

The system mainly uses two entities: User and Location.

User Entity:

- ID - Unique ID of the user.
- Name - User's name.
- Email - User's email address.
- Password - User's password.
- Role - User's role.

Location Entity:

- ID - Unique ID of the location
- User - Links the location to a user
- City - Name of the city
- Country - Name of the country
- Created At - Date the location was added

Relationship:

One user can save many favorite locations. This is a one-to-many relationship.

AI WeatherWise - Database (ER) Diagram

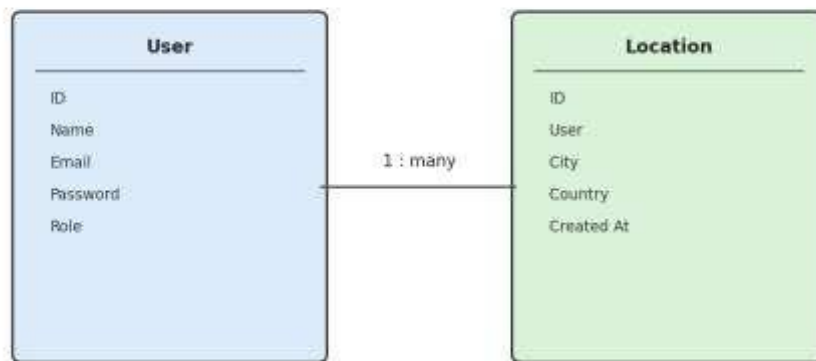


Figure 2: Database (ER) Diagram

Roles and Responsibilities:

Admin:

- Manage user accounts.
- Check API and system status.
- Monitor the database.

Registered User:

- Create and manage favorite locations.
- Request the current weather.
- Get AI-based weather summaries and suggestions.
- Manage personal account details.

User Flow:

- User registers or logs in.
- User saves a favorite city.
- User asks for the current weather.
- User asks for an AI summary or suggestion.
- The system checks the user's login token and shows the result.

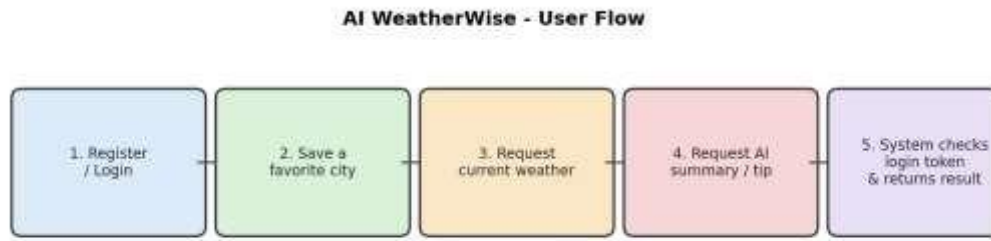


Figure 3: User Flow

MVC Architecture:

Model:

The Model manages the data structure and database information. Mongoose is used to create schemas and work with MongoDB.

Controller:

The Controller receives requests, processes them, and sends back the result.

View / API Layer:

This project is mainly a backend application. Instead of a normal web page, APIs are used to talk to the application.

Project Setup:

Creating the Project:

- Create a folder named AI WeatherWise.
- Open the folder in Visual Studio Code.
- Open the terminal.
- Start (initialize) the Node.js project.

```
npm init-y
```

Install the Required Packages:

```
npm install express mongoose bcryptjs jsonwebtoken cors dotenv multer @google/genai
```

Install Nodemon (for development):

```
npm install --save-dev nodemon.
```

Important Files:

- index.js
- .env
- package.json
- .gitignore

Environment Variables:

The .env file stores important settings such as the port number, MongoDB connection, JWT secret, and Gemini API key.

```
PORT=5000 MONGO_URI=your_mongodb_connection_string JWT_SECRET=your_jwt_secret  
GEMINI_API_KEY=your_gemini_api_key.
```

Backend Development:

The backend is built using Node.js, Express.js, and MongoDB. Routes, controllers, and models are kept in separate files so the code is easy to manage.

The backend provides REST APIs for user login, weather information, AI recommendations, and other tasks.

Main Application File:

The index.js file is the main file of the app. It starts the Express server, loads the settings, connects to MongoDB, adds middleware and starts the API server.

Database Configuration:

MongoDB stores the app's data. Mongoose connects the Node.js app with MongoDB.

Steps:

- Install Mongoose.
- Connect the app to MongoDB.
- Create User and Location schemas.
- Create models and use them in the app.

```
npm install mongoose.
```

Project Execution:

The AI WeatherWise backend can run on a local computer using Node.js. MongoDB must be running and the environment variables must be set.

Run the Project:

- Open the project folder in VS Code.
- Open the terminal.
- Install all packages.
- Start the development server.

```
npm install npm start
```

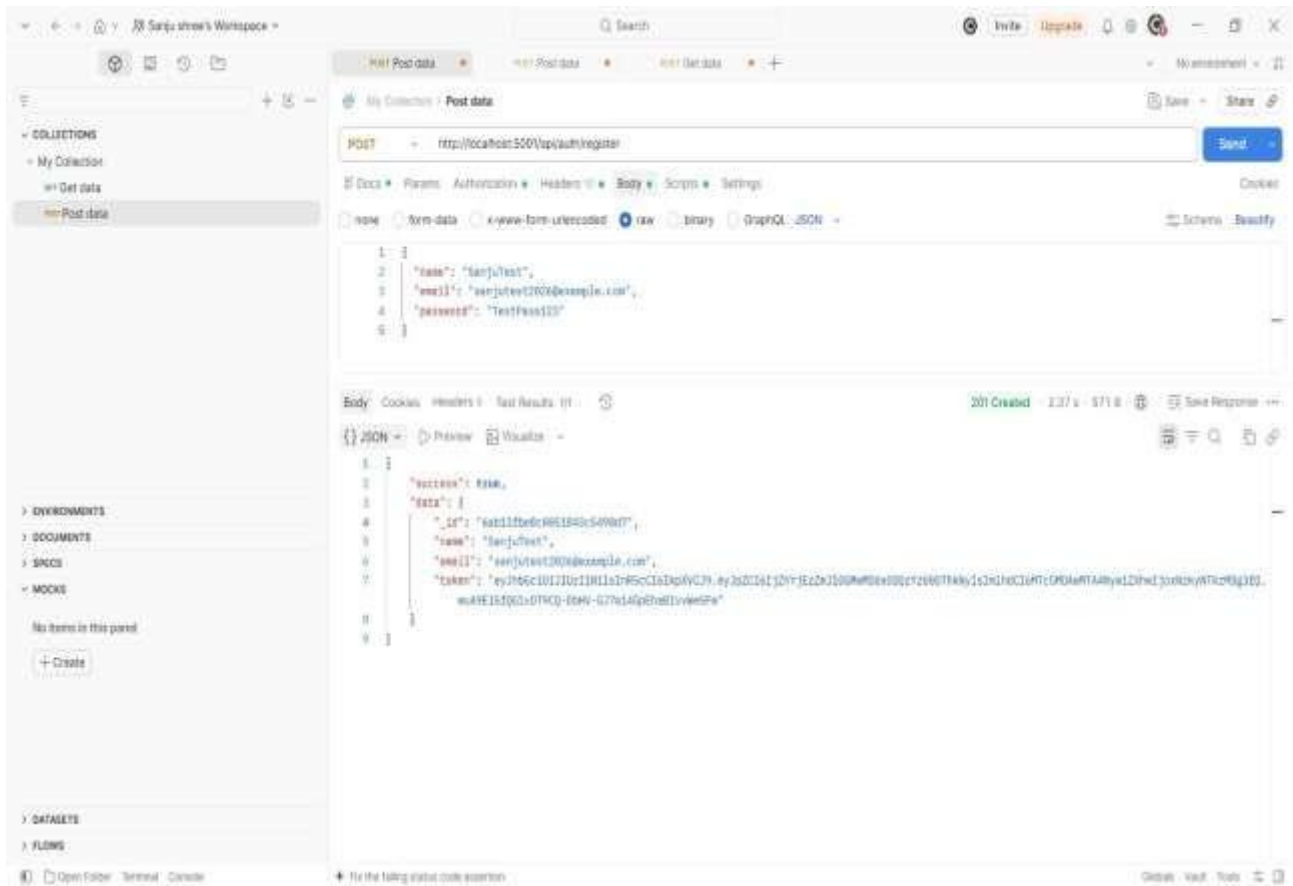
After starting the project, the server connects to MongoDB and waits for API requests.

API Testing:

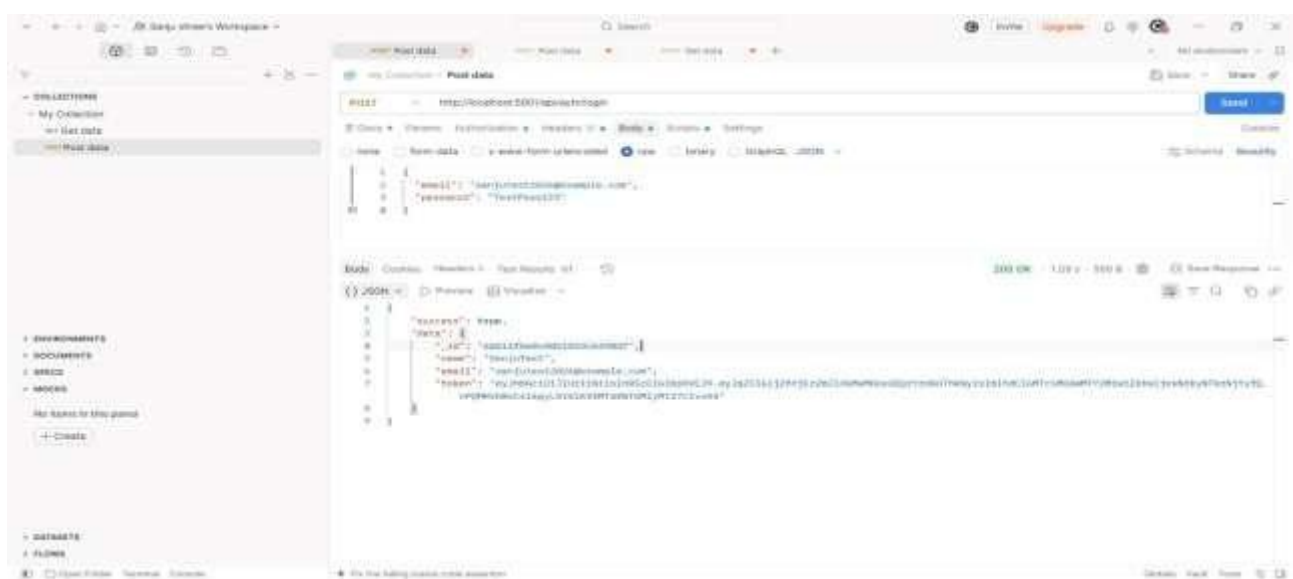
The APIs can be tested using Postman or Thunder Client.

- Register API - Creates a new user.
- Login API - Logs in to the app.
- Weather API - Gets current weather information.
- Recommendation API - Gets AI-based weather suggestions.

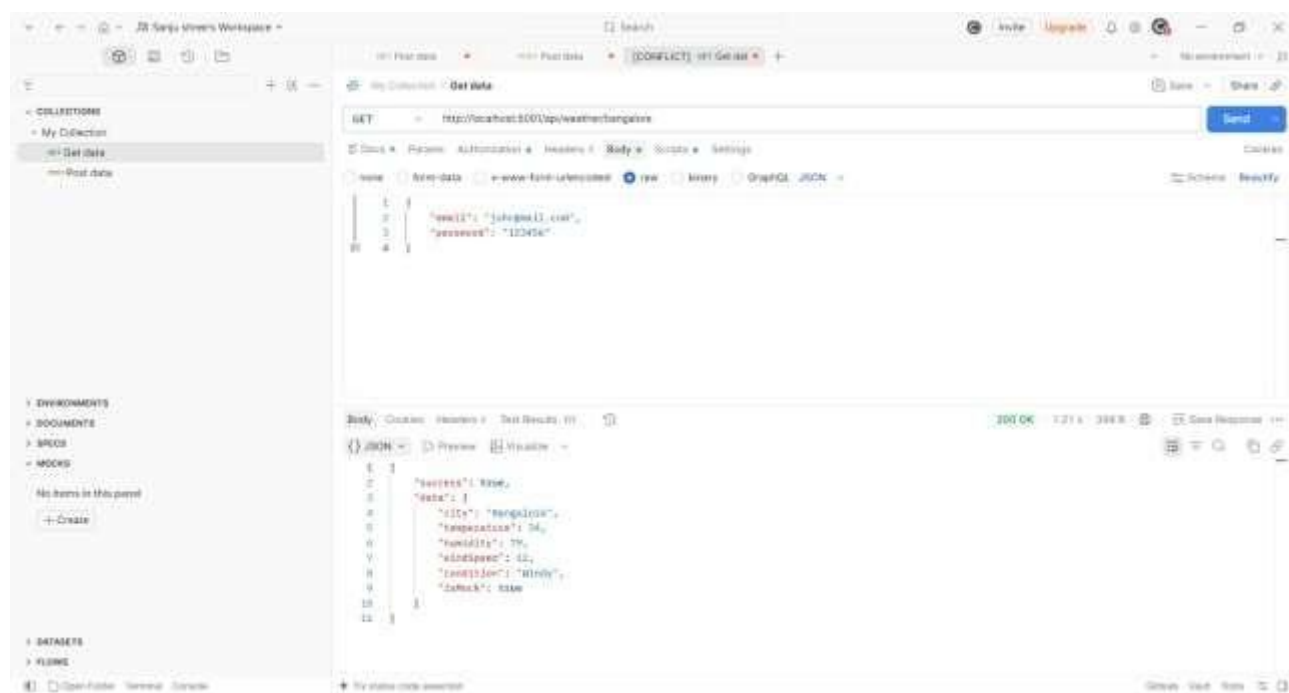
Register API:



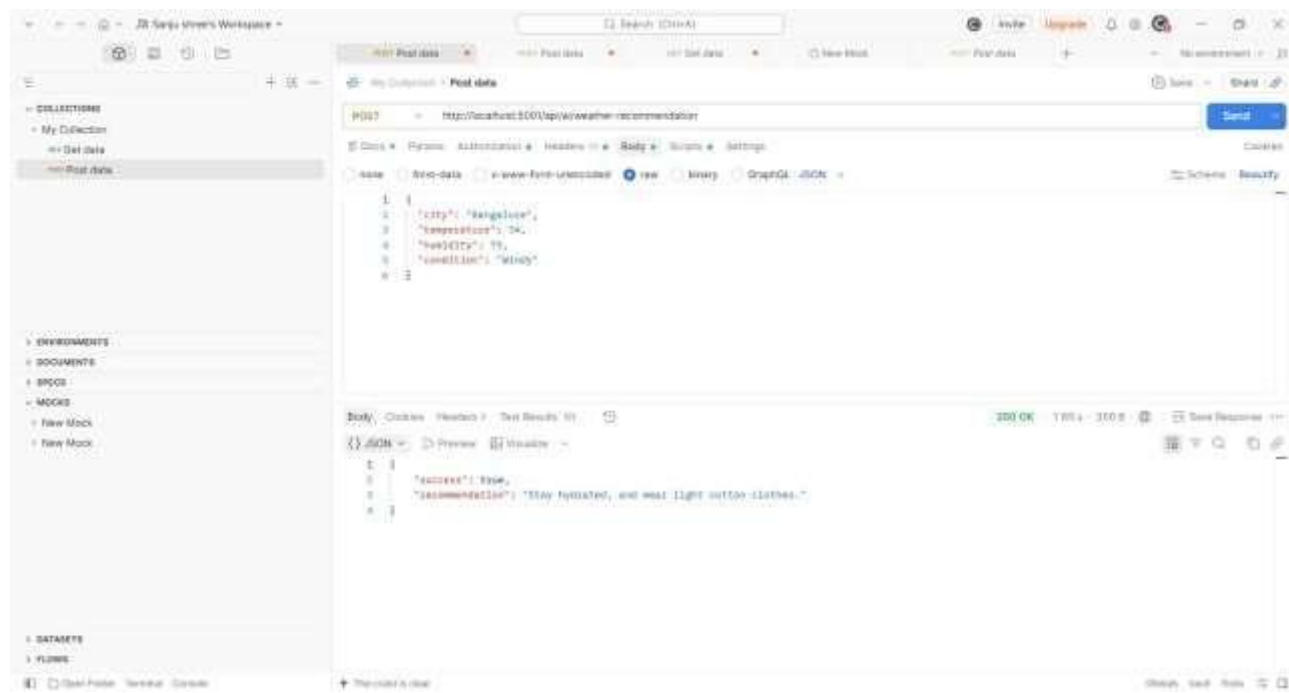
Login API:



Weather API:



Recommendation API:



Demo Video Link:

https://drive.google.com/file/d/1JOV0Wg9h5orOB2ud2qA41DxMW0_HsHbF/view?usp=sharing